

## Processador VLIW

Esse é um relatório de um processador VLIW na arquitetura Sagui de Rabo Longo, da disciplina de Arquitetura de Computadores 2026/1. O trabalho consiste em um processador VLIW no logisim, bem como um código para testes. Alunos: Letícia de Oliveira Santos e Iago Pereira

### 1. Do processador

#### a. Componentes:

- 2 ULAs (unidades lógico - aritméticas)
  - Somador, subtrator, incrementador, OR lógico, NOT lógico, shift left register e shift right register
- Control - unidade que lida com os sinais de controle
- Pc\_Control - unidade que lida com os sinais de controle do PC
- Move - unidade que faz a operação move
- Zero - unidade que determina se um número é zero, se não for, retorna 1
- Banco de Registradores - onde há escrita e leitura dos registradores
- Dual port Ram - memória de dados e instruções
- is\_move: unidade que determina se a instrução é move ou não

#### b. Tabelas de Controle

##### i. Lane 1 LD/ST/MOVE

| Instr  | MemRead | MemWrite | RegWriteLn1 | Mux_L1 |
|--------|---------|----------|-------------|--------|
| ld     | 1       | 0        | 1           | 0      |
| st     | 0       | 1        | 0           | 0      |
| movh   | 0       | 0        | 1           | 1      |
| movl   | 0       | 0        | 1           | 1      |
| outras | 0       | 0        | 0           | 0      |

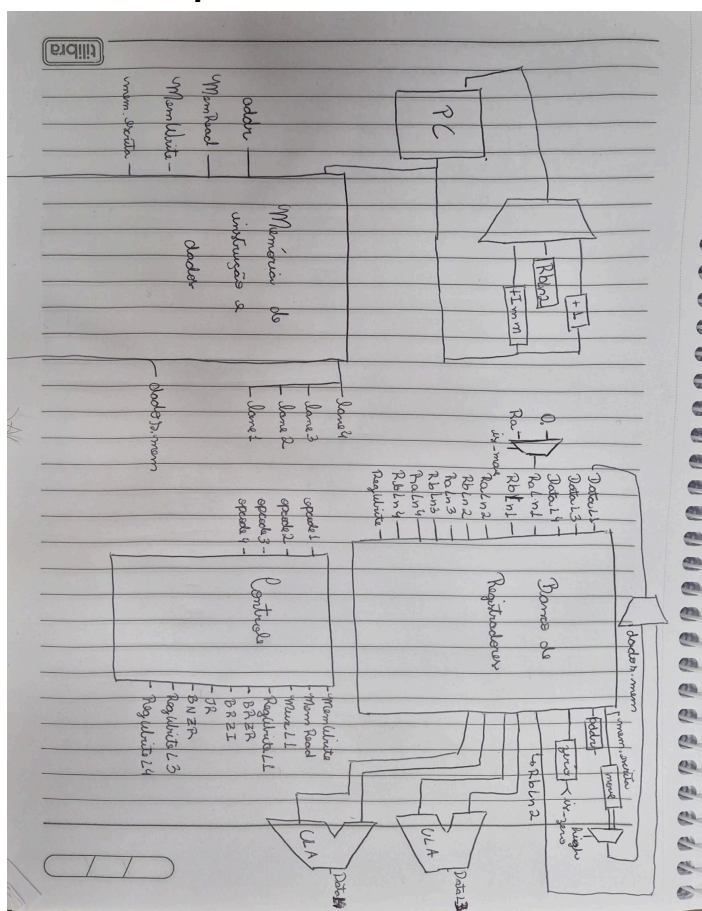
**ii. Lane 2 BR/JUMP**

| <b>Instr</b> | <b>Sig_Brzt</b> | <b>Sig_Brzi</b> | <b>Sig_JR</b> | <b>Sig_Bnzt</b> |
|--------------|-----------------|-----------------|---------------|-----------------|
| brzt         | 1               | 0               | 0             | 0               |
| brzi         | 0               | 1               | 0             | 0               |
| jr           | 0               | 0               | 1             | 0               |
| bnzt         | 0               | 0               | 0             | 1               |
| outras       | 0               | 0               | 0             | 0               |

**iii. Lane 3 e 4 ULA**

| <b>Instr</b> | <b>RegWriteLn3/4</b> | <b>OpUla</b> |
|--------------|----------------------|--------------|
| add          | 1                    | 000          |
| sub          | 1                    | 001          |
| inc          | 1                    | 010          |
| or           | 1                    | 011          |
| not          | 1                    | 100          |
| slr          | 1                    | 101          |
| srr          | 1                    | 110          |
| outras       | 0                    | X            |

### c. Datapath



### d. Novas instruções

#### i. Branch on Not Zero Register - bnzr - 0011

1. **Descrição:** se  $Ra \neq 0$ ,  $PC = Rb$

2. **Motivação:** é útil em laços que decrementam o parâmetro. Ao invés de comparar com 0 em cada laço, ele continua rodando enquanto o parâmetro não é 0

#### ii. Increment - inc - 1010

1. **Descrição:**  $Ra = Ra + 1$

2. **Motivação:** facilita somas com o "imediato" 1

### 2. Programa de Teste

Esse é um programa que realiza a soma de 2 vetores (A e B) e armazena em um vetor R. Os números à esquerda representam as instruções que estão no mesmo "bloco".

00.

L1 (1/4): movh 0  
L2 (2/4): nop  
L3 (3/4): sub R3, R3 // r3 = 0  
L4 (4/4): sub R2, R2 // r2 = 0

01.

L1 (1/4): movl -4 // r0 = 12  
L2 (2/4): nop  
L3 (3/4): sub R1, R1 // r1 = 0 (r1 = i)  
L4 (4/4): nop

02.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): add R3, R0 // r3 = 12 (contador)  
L4 (4/4): nop

03. // > A[i] = i

L1 (1/4): movh 4  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

04.

L1 (1/4): movl 0 r0 = 64 (base do vetor A)  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

05.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): add R0, R1 // r0 = baseA + i  
L4 (4/4): nop

06.

L1 (1/4): st R1, R0 // Grava i na memória, em 64+i  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

07. // > B[i] = i + 20

L1 (1/4): movh 1 //  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

08.

L1 (1/4): movl 4        // r0 = 20  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

09.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): add R2, R0    // r2 = 20  
L4 (4/4): nop

10.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): add R2, R1    // R2 = 20 + i  
L4 (4/4): nop

11.

L1 (1/4): movh 5  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

12.

L1 (1/4): movl 0        //base do vetor b (80)  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

13.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): add R0, R1    // r0 = endereço do B + i  
L4 (4/4): nop

14.

L1 (1/4): st R2, R0    // grava na memória B[i]  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

15.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): sub R2, R2  
L4 (4/4): nop

16.

L1 (1/4): movh 6  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

17.

L1 (1/4): movl 0 //base do vetor R (96)  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

18.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): add R0, R1 // r0 = Base do R + i (endereço)  
L4 (4/4): nop

19.

L1 (1/4): st R2, R0 // grava 0 no primeiro endereço de r  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

20.

L1 (1/4): movh 0 //r0 = 0  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

21.

L1 (1/4): movl 1 // R0 = 1  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

22.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): inc R1 // i++ (ULA 1)  
L4 (4/4): sub R3, R0 // R3-- (ULA 2)

23.

L1 (1/4): movh 0  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

24.

L1 (1/4): movl 3        // Alvo do pulo (03)  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

25.

L1 (1/4): nop  
L2 (2/4): bnzr R3, R0    // Salta se R3 != 0  
L3 (3/4): nop  
L4 (4/4): nop

//reseta contadores

26.

L1 (1/4): movh 0  
L2 (2/4): nop  
L3 (3/4): sub R1, R1    // i = 0  
L4 (4/4): nop

27.

L1 (1/4): movl -4        // R0 = 12  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

28.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): add R3, R0    // R3 = 12 de novo  
L4 (4/4): nop

//soma dos vetores

29. // > Ler A[i]

L1 (1/4): movh 4        // Base A  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

30.

L1 (1/4): movl 0  
L2 (2/4): nop  
L3 (3/4): nop  
L4 (4/4): nop

31.

L1 (1/4): nop  
L2 (2/4): nop  
L3 (3/4): add R0, R1    // Endereço A[i]

L4 (4/4): nop

32.

L1 (1/4): ld R2, R0 // R2 recebe A[i]

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

33. // > Ler B[i]

L1 (1/4): movh 5 // Base B

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

34.

L1 (1/4): movl 0

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

35.

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): add R0, R1 // Endereço B[i]

L4 (4/4): nop

36.

L1 (1/4): ld R0, R0 // R0 recebe B[i]

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

37. // > Fazer a Soma

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): add R2, R0 // R2 = A[i] + B[i]

L4 (4/4): nop

38. // > Gravar em R[i]

L1 (1/4): movh 6 // Base R

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

39.

L1 (1/4): movl 0

L2 (2/4): nop

L3 (3/4): nop



L4 (4/4): nop

40.

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): add R0, R1 // Endereço R[i]

L4 (4/4): nop

41.

L1 (1/4): st R2, R0 // Grava resultado na RAM

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

42. // > Controle do Laço

L1 (1/4): movh 0

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

43.

L1 (1/4): movl 1 // R0 = 1

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

44.

L1 (1/4): nop

L2 (2/4): nop

L3 (3/4): inc R1 // i++ (ULA 1)

L4 (4/4): sub R3, R0 // R3-- (ULA 2)

45.

L1 (1/4): movh 1

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

46.

L1 (1/4): movl -3 // Alvo do pulo (29 = 0x1D)

L2 (2/4): nop

L3 (3/4): nop

L4 (4/4): nop

47.

L1 (1/4): nop

L2 (2/4): bnzr R3, R0 // Salta se R3 != 0

L3 (3/4): nop

*L4 (4/4): nop*

48.

*L1 (1/4): nop*

*L2 (2/4): nop*

*L3 (3/4): sub R0, R0    // Garante R0 = 0*

*L4 (4/4): nop*

49.

*L1 (1/4): nop*

*L2 (2/4): brzi 0        // Pula para a própria linha*

*L3 (3/4): nop*

*L4 (4/4): nop*